Отчёт по лабораторной работе №11

Настройка безопасного удалённого доступа по протоколу SSH

Цыкунова Екатерина Михайловна

Содержание

1 Цель работы **5**

2 Выполнение лабораторной работы **6**
2.1 Запрет удалённого доступа по SSH для пользователя root 6
2.2 Ограничение списка пользователей для удалённого доступа по SSH 8
2.3 Настройка дополнительных портов для удалённого доступа по SSH 11
2.4 Настройка удалённого доступа по SSH по ключу 16
2.5 Организация SSH-туннеля и перенаправление TCP-портов 18
2.6 Запуск консольных приложений через SSH 20
2.7 Запуск графических приложений через SSH (X11Forwarding) . . . 22
2.8 Внесение изменений в настройки внутреннего окружения виртуальной машины 23

3 Вывод **25**

4 Контрольные вопросы **26**

5 Список литературы **29**

Список иллюстраций

2.1 Попытка подключения к серверу по SSH от имени root 7
2.2 Запрет удалённого доступа по SSH для root 7
2.3 Проверка запрета подключения пользователя root 8
2.4 Успешное подключение к серверу от имени пользователя emcikunova 8
2.5 Ограничение доступа по SSH пользователем vagrant 9
2.6 Отказ в подключении пользователя emcikunova 9
2.7 Добавление пользователя emcikunova в список разрешённых . . . 10
2.8 Проверка успешного подключения после изменения AllowUsers . 11
2.9 Добавление портов 22 и 2022 в конфигурацию SSH 11
2.10 Ошибка привязки SSH-сервера к порту 2022 12
2.11 Сообщения SELinux о блокировке порта 2022 13
2.12 Настройка SELinux, межсетевого экрана и проверка работы SSH . 14
2.13 Проверка SSH-соединения через порт 22 15
2.14 Проверка SSH-соединения через порт 2022 15
2.15 Разрешение аутентификации SSH по открытому ключу 16
2.16 Генерация пары SSH-ключей на клиенте 17
2.17 Копирование открытого ключа и проверка подключения без пароля 18
2.18 Создание SSH-туннеля и проверка TCP-соединений 19
2.19 Проверка работы SSH-туннеля в браузере 20
2.20 Удалённое выполнение команд hostname и ls через SSH 21
2.21 Просмотр почтового ящика сервера через SSH 21
2.22 Разрешение перенаправления X11 в конфигурации SSH 22
2.23 Попытка запуска Firefox через SSH с перенаправлением X11 . . . 23
2.24 Создание каталога конфигурации SSH и исполняемого скрипта . 24
2.25 Скрипт автоматической настройки SSH-сервера 24

Список таблиц

1 Цель работы

Приобретение практических навыков по настройке удалённого доступа к серверу с помощью SSH.

2 Выполнение лабораторной работы

2.1 Запрет удалённого доступа по SSH для пользователя root

На виртуальной машине server.emcikunova.net перехожу в режим суперпользователя и при необходимости задаю пароль для пользователя root:

```
sudo -i
passwd root
```

Для мониторинга системных событий в дополнительном терминале запускаю команду:

```
journalctl -x -f
```

На виртуальной машине client.emcikunova.net выполняю попытку подключения к серверу по SSH от имени пользователя root:

```
ssh root@server.emcikunova.net
```

При первом подключении система запрашивает подтверждение подлинности сервера и добавляет его ключ в файл known_hosts. После трёх попыток ввода пароля соединение завершается сообщением Permission denied. Это означает, что аутентификация пользователя root не прошла (см. рис. Рисунок 2.1).



Рисунок 2.1: Попытка подключения к серверу по SSH от имени root

На сервере открываю конфигурационный файл SSH-сервера `/etc/ssh/sshd_config` и устанавливаю параметр:

```
PermitRootLogin no
```

Данный параметр запрещает удалённое подключение по SSH непосредственно от имени пользователя `root` (см. рис. Рисунок 2.2).

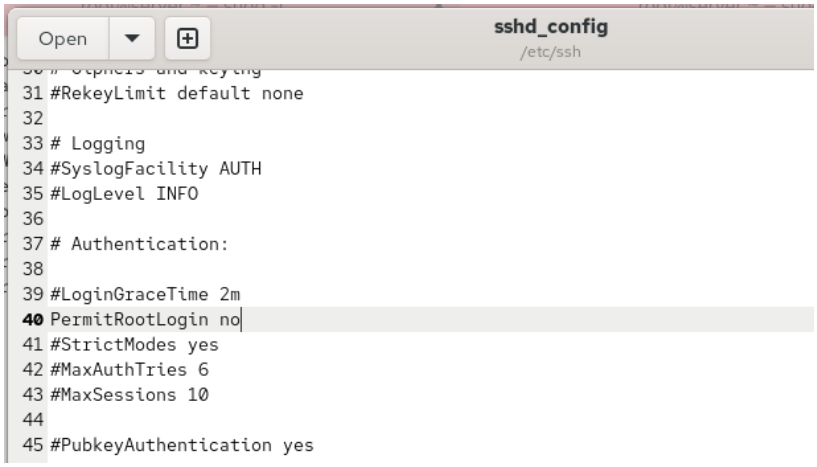


Рисунок 2.2: Запрет удалённого доступа по SSH для root

Для применения изменений перезапускаю службу SSH:

```
systemctl restart sshd
```

Повторно пытаюсь подключиться к серверу от имени `root`:

```
ssh root@server.emcikunova.net
```

Сервер снова отклоняет попытки аутентификации и выводит сообщение `Permission denied`. Теперь в конфигурации явно установлен запрет удалённого входа для пользователя `root` (см. рис. Рисунок 2.3).



Рисунок 2.3: Проверка запрета подключения пользователя root

2.2 Ограничение списка пользователей для удалённого доступа по SSH

На клиенте проверяю возможность подключения к серверу от имени пользователя `emcikunova`:

```
ssh emcikunova@server.emcikunova.net
```

После ввода пароля подключение успешно устанавливается. На экране появляется приглашение командной строки пользователя на сервере. Завершаю сеанс командой `exit` (см. рис. Рисунок 2.4).

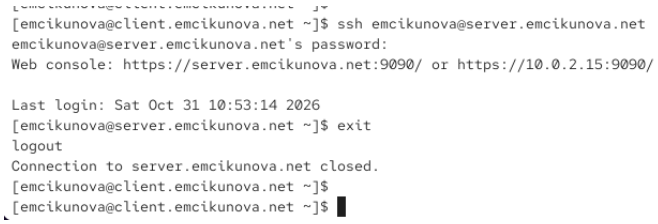


Рисунок 2.4: Успешное подключение к серверу от имени пользователя emcikunova

На сервере открываю файл `/etc/ssh/sshd_config` и добавляю директиву:

```
AllowUsers vagrant
```

Параметр `AllowUsers` ограничивает список пользователей, которым разрешено подключаться по SSH. В данном случае доступ предоставляется только пользователю `vagrant` (см. рис. Рисунок 2.5).

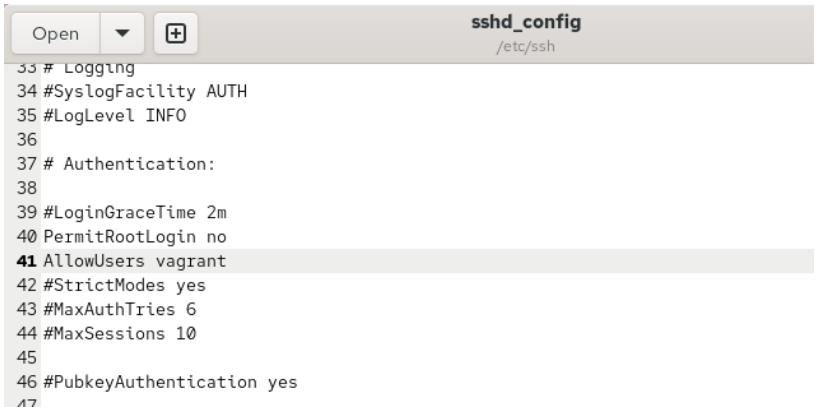


Рисунок 2.5: Ограничение доступа по SSH пользователем vagrant

Перезапускаю службу SSH:

```
systemctl restart sshd
```

С клиентской машины повторно выполняю подключение:

```
ssh emcikunova@server.emcikunova.net
```

После ввода пароля сервер отклоняет подключение с сообщением `Permission denied`, поскольку пользователь `emcikunova` отсутствует в списке разрешённых пользователей (см. рис. Рисунок 2.6).

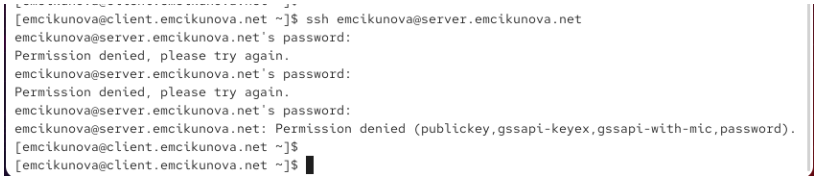


Рисунок 2.6: Отказ в подключении пользователя emcikunova

Для предоставления доступа пользователю emcikunova изменяю директиву в конфигурационном файле /etc/ssh/sshd_config:

```
AllowUsers vagrant emcikunova
```

Теперь подключение по SSH разрешено пользователям vagrant и emcikunova, а остальные пользователи не входят в список разрешённых (см. рис. Рисунок 2.7).

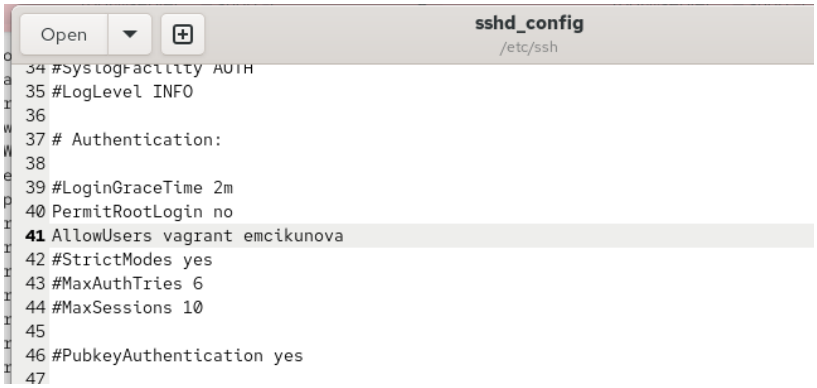


Рисунок 2.7: Добавление пользователя emcikunova в список разрешённых

Перезапускаю службу:

```
systemctl restart sshd
```

Повторно подключаюсь к серверу:

```
ssh emcikunova@server.emcikunova.net
```

Подключение проходит успешно. При входе система сообщает о предыдущих неудачных попытках аутентификации. После проверки завершаю SSH-сеанс командой exit (см. рис. Рисунок 2.8).

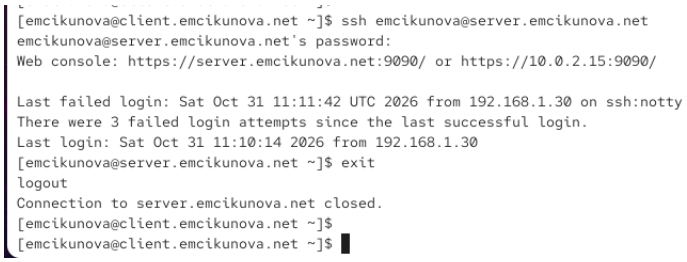


Рисунок 2.8: Проверка успешного подключения после изменения AllowUsers

2.3 Настройка дополнительных портов для удалённого доступа по SSH

На сервере открываю файл `/etc/ssh/sshd_config` и добавляю настройки для использования двух TCP-портов:

```
Port 22
Port 2022
```

Порт 22 используется для стандартных SSH-соединений, а порт 2022 добавляется в качестве альтернативного. Сохранение стандартного порта позволяет продолжить подключение при возникновении проблем с дополнительным портом (см. рис. Рисунок 2.9).

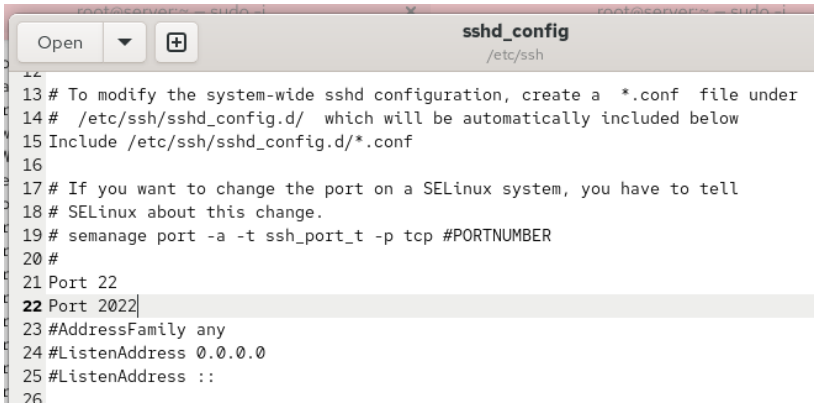


Рисунок 2.9: Добавление портов 22 и 2022 в конфигурацию SSH

Перезапускаю службу SSH и проверяю её расширенный статус:

```
systemctl restart sshd
systemctl status -l sshd
```

В выводе команда `systemctl status` показывает, что служба `sshd` находится в состоянии `active (running)`. Однако при попытке прослушивания порта `2022` появляется ошибка `Bind to port 2022 ... failed: Permission denied`.

При этом порт 22 продолжает прослушиваться, поэтому SSH-сервер остаётся доступным через стандартный порт (см. рис. Рисунок 2.10).

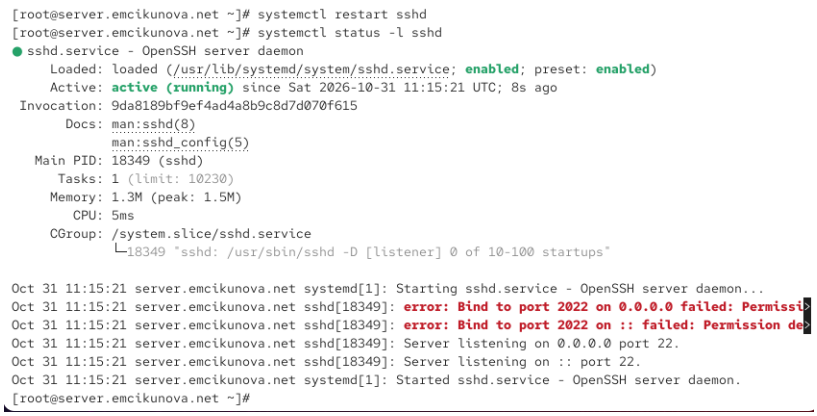


Рисунок 2.10: Ошибка привязки SSH-сервера к порту 2022

В терминале с мониторингом системных событий изучаю сообщения SELinux.

В журнале появляется сообщение о том, что SELinux блокирует процессу `/usr/sbin/sshd` операцию `name_bind` для TCP-порта `2022`. Это происходит потому, что данный порт ещё не разрешён политикой SELinux для службы SSH (см. рис. Рисунок 2.11).

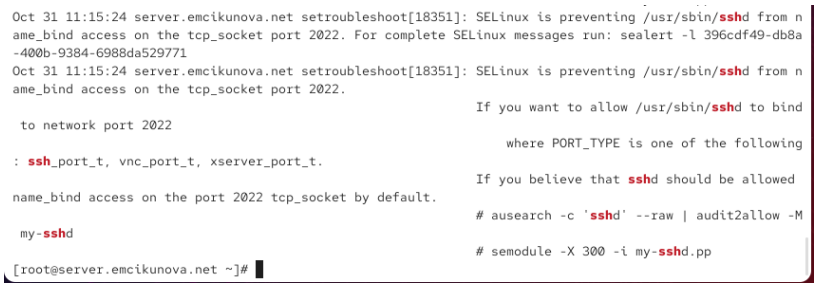


Рисунок 2.11: Сообщения SELinux о блокировке порта 2022

Для разрешения использования порта 2022 службой SSH добавляю его в политику SELinux с типом ssh_port_t:

```
semanage port -a -t ssh_port_t -p tcp 2022
```

Затем открываю TCP-порт 2022 в межсетевом экране для текущей и постоянной конфигурации:

```
firewall-cmd --add-port=2022/tcp
firewall-cmd --add-port=2022/tcp --permanent
```

Обе команды завершаются сообщением success.

После изменения настроек перезапускаю SSH-сервер и проверяю состояние службы:

```
systemctl restart sshd
systemctl status -l sshd
```

Теперь служба успешно запускается и прослушивает оба TCP-порта: 22 и 2022. Это подтверждается сообщениями Server listening on ... port 2022 и Server listening on ... port 22 (см. рис. Рисунок 2.12).

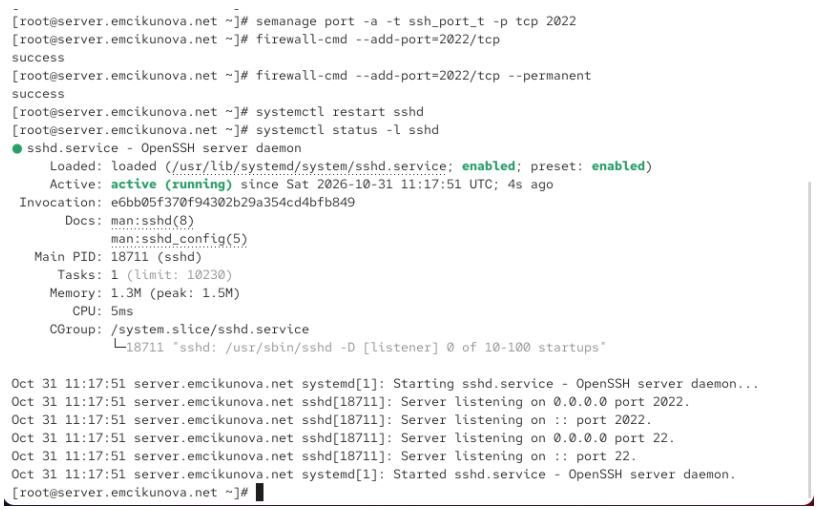


Рисунок 2.12: Настройка SELinux, межсетевого экрана и проверка работы SSH

С клиентской машины проверяю подключение через стандартный порт 22:

```
ssh emcikunova@server.emcikunova.net
```

После успешного подключения получаю права суперпользователя:

```
sudo -i
```

Приглашение командной строки изменяется на `root@server.emcikunova.net`, что подтверждает получение административных прав.

Завершаю сеансы пользователя `root` и пользователя `emcikunova`:

```
logout
logout
```

Подключение через стандартный SSH-порт работает корректно (см. рис. Рисунок 2.13).

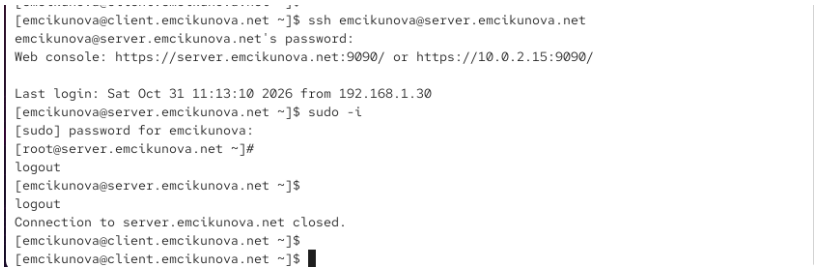


Рисунок 2.13: Проверка SSH-соединения через порт 22

Далее проверяю возможность подключения через дополнительный порт 2022:

```
ssh -p 2022 emcikunova@server.emcikunova.net
```

Соединение устанавливается успешно. Получаю права суперпользователя командой:

```
sudo -i
```

После проверки завершаю оба сеанса:

```
logout
logout
```

Успешное подключение подтверждает, что дополнительный порт 2022 разрешён политикой SELinux, открыт в межсетевом экране и прослушивается службой sshd (см. рис. Рисунок 2.14).

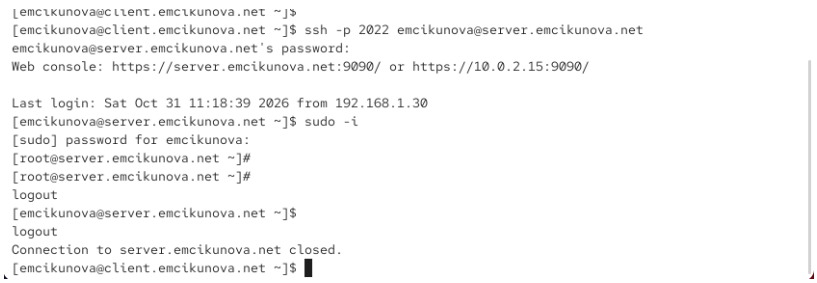


Рисунок 2.14: Проверка SSH-соединения через порт 2022

2.4 Настройка удалённого доступа по SSH по ключу

На сервере открываю конфигурационный файл `/etc/ssh/sshd_config` и включаю аутентификацию по открытому ключу:

```
PubkeyAuthentication yes
```

Параметр разрешает пользователям проходить аутентификацию с помощью криптографической пары ключей вместо ввода пароля учётной записи (см. рис. Рисунок 2.15).

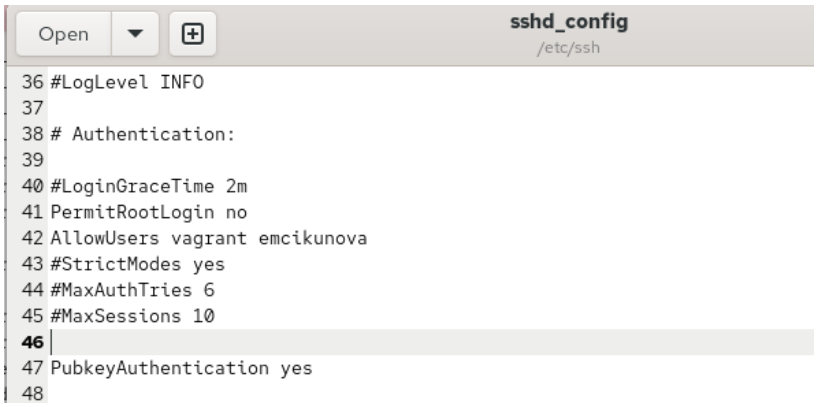


Рисунок 2.15: Разрешение аутентификации SSH по открытому ключу

После сохранения изменений перезапускаю SSH-сервер:

```
systemctl restart sshd
```

На виртуальной машине `client.emcikunova.net` от имени пользователя `emcikunova` создаю пару SSH-ключей:

```
ssh-keygen
```

При создании ключей принимаю стандартный путь сохранения и оставляю кодовую фразу пустой.

В результате создаётся пара ключей алгоритма ED25519:

- `~/.ssh/id_ed25519` — закрытый ключ;
- `~/.ssh/id_ed25519.pub` — открытый ключ.

Команда также выводит отпечаток открытого ключа и его графическое представление `randomart` (см. рис. Рисунок 2.16).

```
[emcikunova@client.emcikunova.net ~]$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/emcikunova/.ssh/id_ed25519):
Enter passphrase for "/home/emcikunova/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/emcikunova/.ssh/id_ed25519
Your public key has been saved in /home/emcikunova/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:If2i3c67Ay+IdhRE2xHE+Ry1eAZK6Fn4YH3xbUnoRiM emcikunova@client.emcikunova.net
The key's randomart image is:
+--[ED25519 256]--+
|     ..*=ooo...  |
|      *==oE+++ . |
|     +o==+.==.+  |
|      +..oooo.   |
|       .S ..     |
|      .o.o       |
|     o...o.      |
|    o o .oo      |
|   . .   .=+     |
+----[SHA256]-----+
[emcikunova@client.emcikunova.net ~]$
[emcikunova@client.emcikunova.net ~]$
```

Рисунок 2.16: Генерация пары SSH-ключей на клиенте

Копирую открытый ключ с клиента на сервер:

```
ssh-copy-id emcikunova@server.emcikunova.net
```

При выполнении команды ввожу пароль пользователя `emcikunova` на сервере. Утилита сообщает об успешном добавлении одного ключа в список разрешённых ключей удалённой учётной записи.

После этого проверяю подключение:

```
ssh emcikunova@server.emcikunova.net
```

Соединение устанавливается без запроса пароля пользователя. Это подтверждает успешную настройку аутентификации по SSH-ключу.

Завершаю подключение командой `logout` (см. рис. Рисунок 2.17).

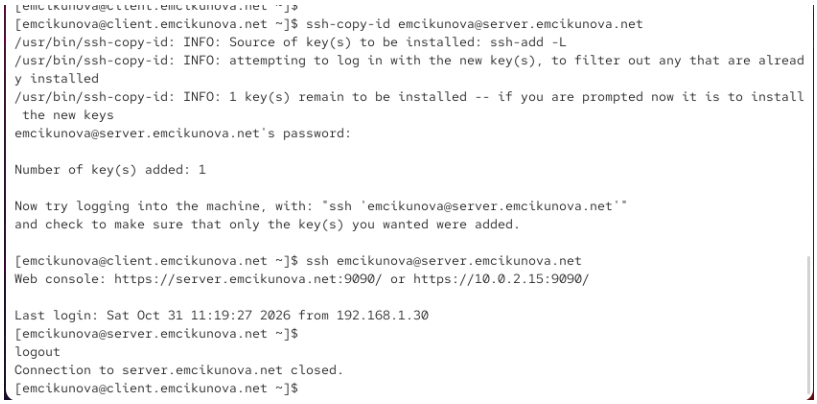


Рисунок 2.17: Копирование открытого ключа и проверка подключения без пароля

2.5 Организация SSH-туннеля и перенаправление TCP-портов

На клиентской машине проверяю существующие TCP-соединения:

```
lsof | grep TCP
```

Затем создаю SSH-туннель, перенаправляющий подключения к локальному TCP-порту `8080` на порт `80` удалённого сервера:

```
ssh -fNL 8080:localhost:80 emcikunova@server.emcikunova.net
```

В данной команде:

- `-f` — переводит процесс SSH в фоновый режим;
- `-N` — запрещает выполнение удалённой команды;
- `-L` — настраивает локальное перенаправление TCP-порта;
- `8080:localhost:80` — связывает локальный порт `8080` с портом `80` на удалённой стороне SSH-соединения.

После создания туннеля повторно проверяю TCP-соединения:

```
lsof | grep TCP
```

В выводе появляются процессы `ssh`, использующие локальный адрес `localhost` и состояние `LISTEN`. Это означает, что SSH-клиент ожидает подключения к локальному перенаправленному порту.

Также отображается установленное SSH-соединение между клиентской и серверной машинами (см. рис. Рисунок 2.18).

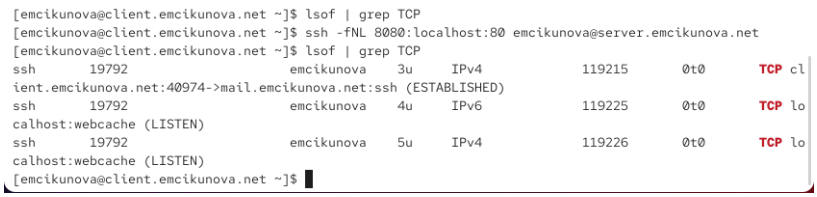


Рисунок 2.18: Создание SSH-туннеля и проверка TCP-соединений

На клиенте открываю браузер Firefox и перехожу по адресу:

```
http://localhost:8080
```

В браузере отображается страница с сообщением:

```
Welcome to the server.emcikunova.net server.
```

Появление страницы подтверждает, что HTTP-запросы с локального порта `8080` успешно передаются через SSH-туннель на TCP-порт `80` удалённого сервера (см. рис. Рисунок 2.19).

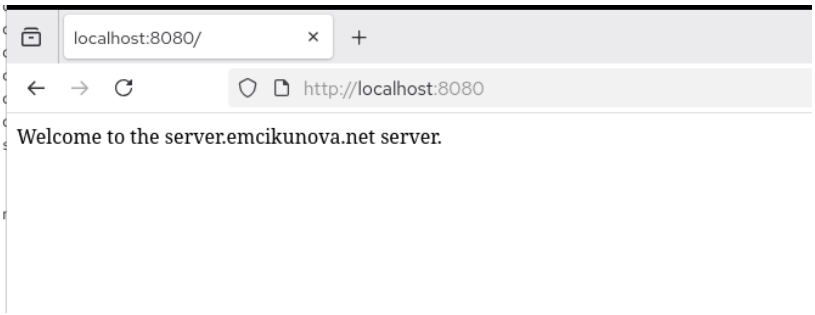


Рисунок 2.19: Проверка работы SSH-туннеля в браузере

2.6 Запуск консольных приложений через SSH

На клиентской машине открываю терминал от имени пользователя `emcikunova`.

Для выполнения команд на удалённом сервере использую SSH без открытия интерактивной командной оболочки.

Сначала получаю имя удалённого узла:

```
ssh emcikunova@server.emcikunova.net hostname
```

Команда возвращает:

```
server.emcikunova.net
```

Затем запрашиваю подробный список файлов домашнего каталога пользователя на сервере:

```
ssh emcikunova@server.emcikunova.net ls -Al
```

В терминале отображается список файлов и каталогов с их правами доступа, владельцами, размерами и датами изменения.

Среди отображаемых объектов присутствуют `.bash_history`, `.bash_profile`, `.bashrc`, `.config`, `Documents`, `Downloads` и другие каталоги (см. рис. Рисунок 2.20).

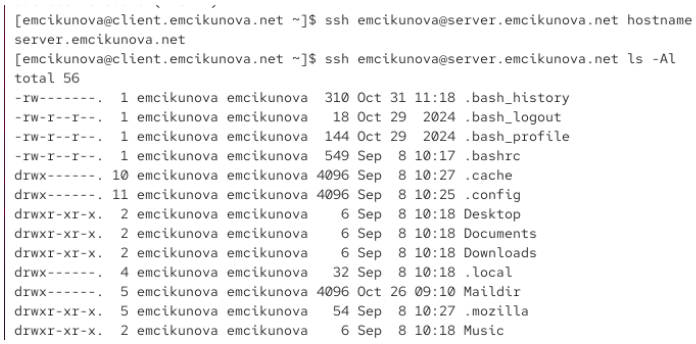


Рисунок 2.20: Удалённое выполнение команд hostname и ls через SSH

Для просмотра почтового ящика на сервере запускаю консольную почтовую программу `mail` через SSH:

```
ssh emcikunova@server.emcikunova.net MAIL=~/Maildir mail
```

В выводе отображается почтовый ящик `/home/emcikunova/Maildir`, содержащий три сообщения, включая письма с темами `test1`, `LMTP test` и `test2`.

После просмотра списка сообщений завершаю работу почтовой программы командой `q`.

Полученные результаты подтверждают возможность удалённого запуска консольных приложений и получения их вывода через SSH (см. рис. Рисунок 2.21).

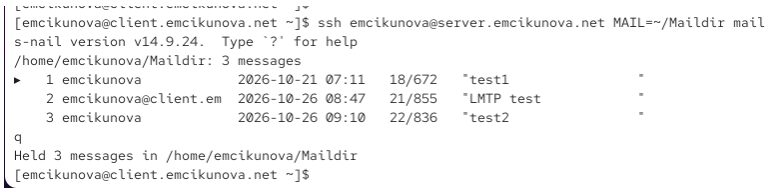


Рисунок 2.21: Просмотр почтового ящика сервера через SSH

2.7 Запуск графических приложений через SSH (X11Forwarding)

На сервере открываю конфигурационный файл `/etc/ssh/sshd_config` и разрешаю перенаправление графического интерфейса X11:

```
X11Forwarding yes
```

Данный параметр позволяет передавать графический интерфейс приложения, запущенного на удалённом сервере, на клиентскую машину при наличии настроенной графической среды и поддержки X11 (см. рис. Рисунок 2.22).

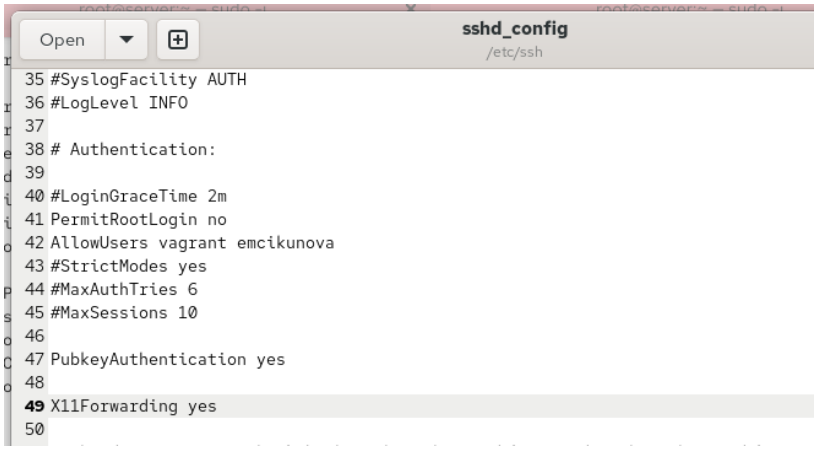


Рисунок 2.22: Разрешение перенаправления X11 в конфигурации SSH

Перезапускаю службу SSH:

```
systemctl restart sshd
```

На клиенте пытаюсь запустить браузер Firefox на удалённом сервере с перенаправлением X11:

```
ssh -YC emcikunova@server.emcikunova.net firefox
```

В результате выполнения команды появляются сообщения:

```
Warning: No xauth data; using fake authentication data for X11 forwarding.
X11 forwarding requested but failed on channel 0
Error: no DISPLAY environment variable specified
```

Сообщения указывают на неудачную настройку графического перенаправления. Переменная окружения `DISPLAY` не задана, поэтому Firefox не может подключиться к графическому дисплею.

Несмотря на включённый параметр `X11Forwarding yes` на сервере, запуск графического приложения не состоялся. Для успешной работы необходимо дополнительно проверить доступность графического X11-дисплея на клиенте и настройки X11-аутентификации (см. рис. Рисунок 2.23).

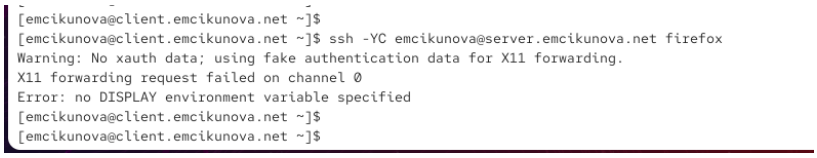


Рисунок 2.23: Попытка запуска Firefox через SSH с перенаправлением X11

2.8 Внесение изменений в настройки внутреннего окружения виртуальной машины

На виртуальной машине `server.emcikunova.net` создаю каталог для хранения конфигурации SSH и исполняемый скрипт `ssh.sh`, который будет использоваться для автоматической настройки сервера (см. рис. Рисунок 2.24).

```
[root@server.emcikunova.net ~]#
[root@server.emcikunova.net ~]# cd /vagrant/provision/server/
[root@server.emcikunova.net server]# mkdir -p /vagrant/provision/server/ssh/etc/ssh
[root@server.emcikunova.net server]# cp -R /etc/ssh/sshd_config /vagrant/provision/server/ssh/etc/ssh/
[root@server.emcikunova.net server]# cd /vagrant/provision/server/
[root@server.emcikunova.net server]# touch ssh.sh
[root@server.emcikunova.net server]# chmod +x ssh.sh
[root@server.emcikunova.net server]#
```

Рисунок 2.24: Создание каталога конфигурации SSH и исполняемого скрипта

В скрипте `ssh.sh` прописываю команды копирования конфигурационных файлов, настройки межсетевого экрана и SELinux, а также перезапуска службы `sshd` (см. рис. Рисунок 2.25).

Vagrantfile | mail.sh | mail.sh | ssh.sh

```
1  #!/bin/bash
2  echo "Provisioning script $0"
3  echo "Copy configuration files"
4  cp -R /vagrant/provision/server/ssh/etc/* /etc
5  restorecon -vR /etc
6  echo "Configure firewall"
7  firewall-cmd --add-port=2022/tcp
8  firewall-cmd --add-port=2022/tcp --permanent
9  echo "Tuning SELinux"
10 semanage port -a -t ssh_port_t -p tcp 2022
11 echo "Restart sshd service"
12 systemctl restart sshd
13
```

Рисунок 2.25: Скрипт автоматической настройки SSH-сервера

3 Вывод

В ходе лабораторной работы выполнена настройка удалённого доступа к серверу `server.emcikunova.net` посредством SSH.

Запрещён непосредственный SSH-доступ для пользователя `root`, ограничен список разрешённых пользователей учётными записями `vagrant` и `emcikunova`. Настроена работа SSH-сервера через стандартный порт `22` и дополнительный порт `2022` с учётом правил SELinux и межсетевого экрана.

Настроена аутентификация по открытому SSH-ключу, позволяющая пользователю подключаться к серверу без ввода пароля учётной записи. Проверено перенаправление TCP-портов через SSH-туннель и удалённое выполнение консольных команд.

Также включена поддержка X11Forwarding, однако запуск графического приложения не удался из-за проблемы с настройкой графического дисплея. Создан скрипт для автоматизации конфигурации SSH-сервера средствами Vagrant.

4 Контрольные вопросы

1. Вы хотите запретить удалённый доступ по SSH на сервер пользователю root и разрешить доступ пользователю alice. Как это сделать?

Для ограничения удалённого доступа необходимо открыть конфигурационный файл `/etc/ssh/sshd_config` и установить параметры:

```
PermitRootLogin no
AllowUsers alice
```

Параметр `PermitRootLogin no` запрещает подключение по SSH пользователю `root`, а `AllowUsers alice` разрешает доступ только пользователю `alice`.

После сохранения изменений необходимо перезапустить службу SSH:

```
systemctl restart sshd
```

2. Как настроить удалённый доступ по SSH через несколько портов? Для чего это может потребоваться?

Для настройки SSH через несколько портов необходимо в файле `/etc/ssh/sshd_config` указать:

```
Port 22
Port 2022
```

После этого необходимо разрешить дополнительный порт в SELinux и межсетевом экране, а затем перезапустить службу `sshd`.

Использование нескольких портов позволяет организовать альтернативные способы подключения к серверу, сохранить доступ при изменении конфигурации и разделить подключения для разных сетевых условий.

3. Какие параметры используются для создания туннеля SSH, когда команда ssh устанавливает фоновое соединение и не ожидает какой-либо конкретной команды?

Для создания SSH-туннеля используются параметры:

- `-f` — переводит SSH-соединение в фоновый режим.
- `-N` — запрещает выполнение удалённой команды.
- `-L` — задаёт локальное перенаправление TCP-порта.

Пример команды:

```
ssh -fNL 8080:localhost:80 user@server.example.com
```

Команда создаёт фоновое SSH-соединение и перенаправляет подключения с локального порта `8080` на порт `80` удалённого сервера.

4. Как настроить локальную переадресацию с локального порта 5555 на порт 80 сервера server2.example.com?

Для настройки локальной переадресации необходимо выполнить команду:

```
ssh -fNL 5555:localhost:80 user@server2.example.com
```

После установки SSH-соединения обращения к локальному адресу `localhost:5555` будут перенаправляться на порт `80` сервера `server2.example.com`.

Для проверки можно открыть в браузере адрес:

```
http://localhost:5555
```

5. Как настроить SELinux, чтобы позволить SSH связываться с портом 2022?

Для разрешения использования порта `2022` службой SSH необходимо выполнить команду:

```
semanage port -a -t ssh_port_t -p tcp 2022
```

Параметр `-a` добавляет порт, `-t ssh_port_t` задаёт тип SELinux для SSH, а `-p tcp` указывает протокол TCP.

После изменения политики SELinux необходимо перезапустить службу:

```
systemctl restart sshd
```

Это позволит SSH-серверу прослушивать порт 2022, если он указан в конфигурации `sshd`.

6. Как настроить межсетевой экран на сервере, чтобы разрешить входящие подключения по SSH через порт 2022?

Для разрешения входящих TCP-соединений через порт 2022 необходимо выполнить команды:

```
firewall-cmd --add-port=2022/tcp
firewall-cmd --add-port=2022/tcp --permanent
```

Первая команда открывает порт в текущей конфигурации межсетевого экрана, а вторая сохраняет правило для последующих запусков системы.

Проверить открытые порты можно командой:

```
firewall-cmd --list-ports
```

После настройки межсетевого экрана и SSH-сервера подключение через дополнительный порт выполняется командой:

```
ssh -p 2022 user@server.example.com
```

5 Список литературы

1. OpenSSH. sshd_config — официальное руководство по настройке SSH-сервера, ограничению доступа пользователей, запрету входа root и настройке дополнительных портов.

2. OpenSSH. ssh — руководство по использованию SSH-клиента, удалённому выполнению команд, перенаправлению TCP-портов и запуску графических приложений через X11Forwarding.

3. OpenSSH. ssh-keygen — официальная документация по генерации криптографических ключей и настройке аутентификации SSH с использованием открытого и закрытого ключей.

4. Red Hat. Securing Networks — руководство по настройке безопасного удалённого доступа через OpenSSH, управлению портами SSH и настройке политик SELinux в Red Hat Enterprise Linux 9.

5. firewalld. Open a Port or Service — официальное руководство по открытию TCP-портов, настройке правил межсетевого экрана и сохранению постоянной конфигурации с помощью firewall-cmd.

6. HashiCorp. Vagrant Shell Provisioner — автоматизация настройки SSH-сервера, выполнения shell-скриптов и применения конфигурационных файлов в виртуальных машинах Vagrant.